Mês 3 — WPT-Piloto: Decomposição e Reconstrução (nov/2026)

Guia de Estudos do Projeto PIBITI

Detecção de Deepfakes de Áudio

20 de setembro de 2026

Sumário

1 O Que o Plano Exige **2**
1.1 Objetivos do Mês 2
1.2 Atividades Previstas 2

2 PyWavelets Do Zero **3**
2.1 O que é PyWavelets e o que é uma Wavelet? 3
2.2 Wavelet-mãe: A Família Daubechies (db4) 3
2.3 A Classe WaveletPacket 3

3 Primeira Decomposição WPT **4**
3.1 Gerando um Sinal Sintético 4
3.2 Aplicando o WaveletPacket 4
3.3 Explorando a Árvore WPT 5

4 Entendendo as Sub-bandas **6**
4.1 A Relação Banda-Frequência 6
4.2 Calculando a Energia das Sub-bandas 6
4.3 O Tamanho da Sub-banda e Efeitos de Borda 6

5 Reconstrução **8**
5.1 Reconstrução Completa 8
5.2 Reconstrução de Sub-bandas Individuais 8
5.3 A Soma das Partes 9

6 Validação da Reconstrução (Crítico) **10**
6.1 A Função de Validação 10
6.2 Bateria de Testes 10

7 Experiência Caixa Preta **12**

8 Funções Reutilizáveis **14**

9 15 Desafios Práticos **15**

10 Referencial Teórico — Template **16**

11 Checklist do Mês 3 **17**

12 Desafio Final do Mês 3 **18**

Capítulo 1

O Que o Plano Exige

1.1 Objetivos do Mês

O objetivo central deste terceiro mês de PIBITI é **fechar a revisão bibliográfica** num formato já utilizável para o relatório parcial/final e **colocar para funcionar a decomposição e reconstrução** via Transformada em Pacotes de Wavelet (WPT).

1.2 Atividades Previstas

- Escrever a versão 1 do Referencial Teórico: conceitos fundamentais sobre wavelets, WPT, descritores de contorno de onda, corpora de áudio (como o ASVspoof) e métricas de avaliação.
- Implementar a decomposição WPT com nível configurável (L) e a reconstrução no domínio do tempo das sub-bandas. Como piloto, usaremos apenas uma wavelet base (`db4`).
- **Validar a reconstrução:** garantir que ela seja quase-perfeita, observando a conservação de energia e o alinhamento (e tamanho) das sub-bandas em relação ao sinal original.

Atenção

Esta é uma fase crítica. O principal **risco técnico** do seu projeto reside aqui. Antes de aplicar qualquer processamento a bases de dados reais, você DEVE testar exaustivamente a reconstrução usando sinais sintéticos conhecidos (ex: senoides puras). Preste muita atenção ao *padding* e ao modo de tratamento de bordas, pois eles introduzem artefatos.

Marco / Entrega

Entregáveis:

1. Referencial teórico redigido.
2. Códigos de decomposição e reconstrução WPT validados (piloto com `db4`).

Capítulo 2

PyWavelets Do Zero

2.1 O que é PyWavelets e o que é uma Wavelet?

O `PyWavelets` (`pywt`) é a principal biblioteca Python para computar transformadas wavelet. Uma wavelet é uma "pequena onda", uma função matemática que é localizada no tempo e na frequência (diferente da Transformada de Fourier, que usa senoides infinitas).

2.2 Wavelet-mãe: A Família Daubechies (db4)

As wavelets são agrupadas em famílias. A `db4` faz parte da família de Daubechies, que é ortogonal e tem suporte compacto (isto é, é zero fora de um pequeno intervalo de tempo). O número "4"indica a ordem.

Listing 2.1: Importando a biblioteca e listando famílias

```
import pywt

print(pywt.families())
```

Listing 2.2: Listando wavelets da familia Daubechies

```
print(pywt.wavelist('db'))
```

2.3 A Classe WaveletPacket

Para a Wavelet Packet Transform (WPT), usamos a classe `WaveletPacket`. Ao contrário da DWT comum (Discrete Wavelet Transform), que divide iterativamente apenas a banda de baixa frequência, a WPT divide ambas as bandas (baixa e alta frequência) formando uma **árvore completa**.

Listing 2.3: Criando um objeto Wavelet genérico

```
w = pywt.Wavelet('db4')
print(f"Nome: {w.name}, Ortogonal: {w.orthogonal}")
```

Capítulo 3

Primeira Decomposição WPT

Neste capítulo vamos gerar um sinal sintético para ver como a decomposição se comporta.

3.1 Gerando um Sinal Sintético

Listing 3.1: Senoide de 440 Hz (Lá)

```
import numpy as np
import matplotlib.pyplot as plt

fs = 16000  # Taxa de amostragem
t = np.arange(0, 1.0, 1/fs)
f = 440     # Frequencia
sinal = np.sin(2 * np.pi * f * t)
```

3.2 Aplicando o WaveletPacket

Listing 3.2: Criando o pacote e decompondo em L=4

```
import pywt

# Criar a arvore WPT
wp = pywt.WaveletPacket(data=sinal, wavelet='db4', mode='symmetric'
    , maxlevel=4)
```

Entendendo os parâmetros:

- `data`: seu sinal de áudio em um vetor Numpy.
- `wavelet='db4'`: A wavelet base.
- `mode='symmetric'`: Como tratar as bordas. Vamos aprofundar isso no próximo capítulo.
- `maxlevel=4`: A *profundidade* da árvore. Teremos $2^4 = 16$ nós nas folhas.

3.3 Explorando a Árvore WPT

Os nós finais contêm as sub-bandas.

Listing 3.3: Acessando os nós no nível 4

```
nos_nivel_4 = wp.get_level(4, order='freq')
print("Quantidade de nos:", len(nos_nivel_4))
```

Dica

Sempre use `order='freq'`. A transformação natural gera as bandas fora de ordem de frequência (conhecido como "sequência de Gray"). Esse parâmetro organiza os nós da menor para a maior frequência.

Listing 3.4: Listando nomes/caminhos dos nós e tamanhos

```
for i, node in enumerate(nos_nivel_4):
    print(f"Banda {i} | Caminho: {node.path} | Tamanho: {len(node.
        data)}")
```

A função `enumerate()` simplesmente gera um índice `i` pareado com cada `node`.

Capítulo 4

Entendendo as Sub-bandas

4.1 A Relação Banda-Frequência

Com $fs = 16000$ Hz, a frequência de Nyquist é 8000 Hz. Se temos $L = 4$, são $2^4 = 16$ bandas. A largura de cada banda é de $8000/16 = 500$ Hz.

- Banda 0: 0 a 500 Hz
- Banda 1: 500 a 1000 Hz
- ...
- Banda 15: 7500 a 8000 Hz

Nossa senoide de 440 Hz deve estar majoritariamente na **Banda 0**.

4.2 Calculando a Energia das Sub-bandas

A energia de um sinal em tempo discreto é a soma dos quadrados de suas amostras.

Listing 4.1: Cálculo de energia por banda

```
energias = []
for node in wp.get_level(4, order='freq'):
    energia = np.sum(node.data ** 2)
    energias.append(energia)

# Encontrar a banda com mais energia
banda_max = np.argmax(energias)
print(f"Banda com mais energia: {banda_max} (esperado 0 para 440Hz)
    ")
```

4.3 O Tamanho da Sub-banda e Efeitos de Borda

Se o sinal original tem 16000 amostras, intuitivamente cada banda teria $16000/16 = 1000$ amostras. Mas o PyWavelets expande o sinal para fazer a convolução (filtros) adequadamente nas pontas (*padding*).

Listing 4.2: Comparando modos de extensão

```
wp_sym = pywt.WaveletPacket(data=sinal, wavelet='db4', mode='
    symmetric', maxlevel=4)
wp_zer = pywt.WaveletPacket(data=sinal, wavelet='db4', mode='zero',
     maxlevel=4)
wp_per = pywt.WaveletPacket(data=sinal, wavelet='db4', mode='
    periodic', maxlevel=4)

print("Symmetric:", len(wp_sym.get_level(4)[0].data))
print("Zero:", len(wp_zer.get_level(4)[0].data))
print("Periodic:", len(wp_per.get_level(4)[0].data))
```

O modo `periodic` geralmente retorna exatamente 1000, mas pode introduzir artefatos se o início e fim do sinal não conectarem suavemente. O `symmetric` adiciona amostras para evitar ruídos de borda, o que é mais seguro.

Capítulo 5

Reconstrução

5.1 Reconstrução Completa

Reconstruir é a função de trazer as sub-bandas de volta ao domínio do tempo com as mesmas dimensões do sinal original.

Listing 5.1: Reconstrução nativa da arvore inteira

```
sinal_reconstruido = wp.reconstruct(update=False)

# Checando o erro
erro = np.max(np.abs(sinal - sinal_reconstruido))
print(f"Erro maximo da reconstrucao total: {erro}")
```

5.2 Reconstrução de Sub-bandas Individuais

E se eu quiser ouvir APENAS a banda 0 no domínio do tempo? Preciso criar uma árvore vazia, copiar APENAS os coeficientes da banda 0 para ela (zerando as outras) e então pedir a reconstrução.

Listing 5.2: Isolando uma sub-banda

```
# Array para guardar a reconstrucao individual de cada banda
reconst_bandas = []

nos = wp.get_level(4, order='freq')

for node in nos:
    # 1. Cria uma WP vazia e identica
    new_wp = pywt.WaveletPacket(data=None, wavelet='db4', mode='
        symmetric')
    # 2. Insere apenas o no atual
    new_wp[node.path] = node.data
    # 3. Reconstroi
    banda_time = new_wp.reconstruct(update=False)
    reconst_bandas.append(banda_time)
```

5.3 A Soma das Partes

Se somarmos todas as reconstruções individuais no tempo, devemos obter o sinal original.

Listing 5.3: Somando e conferindo

```
soma_bandas = np.sum(reconst_bandas, axis=0)
erro_soma = np.max(np.abs(sinal - soma_bandas))
print(f"Erro da soma das bandas: {erro_soma}")
```

Capítulo 6

Validação da Reconstrução (Crítico)

Não acredite até testar exaustivamente.

6.1 A Função de Validação

Listing 6.1: Função mestre de validação

```
def validar_reconstrucao(sinal_orig, sinal_recon):
    # Assegurar mesmo tamanho
    min_len = min(len(sinal_orig), len(sinal_recon))
    sinal_orig = sinal_orig[:min_len]
    sinal_recon = sinal_recon[:min_len]

    erro_max = np.max(np.abs(sinal_orig - sinal_recon))
    mse = np.mean((sinal_orig - sinal_recon)**2)

    energia_orig = np.sum(sinal_orig**2)
    energia_recon = np.sum(sinal_recon**2)
    erro_energia = abs(energia_orig - energia_recon) / energia_orig
        * 100

    print(f"Max Erro: {erro_max:.2e}")
    print(f"MSE:      {mse:.2e}")
    print(f"Erro Eng: {erro_energia:.4f}%")
    return erro_max, mse
```

6.2 Bateria de Testes

Listing 6.2: Testes: Senoides, Ruido e Impulsos

```
# Teste 1: Senoide Pura ja fizemos

# Teste 2: Duas senoides
t = np.arange(0, 1.0, 1/fs)
sinal_2 = np.sin(2 * np.pi * 440 * t) + np.sin(2 * np.pi * 2000 * t
   )

```

```
# Teste 3: Ruido Gaussiano
np.random.seed(42)
sinal_3 = np.random.randn(fs)

# Teste 4: Senoide + Ruido
sinal_4 = sinal_2 + 0.5 * sinal_3

# Teste 5: Impulso
sinal_5 = np.zeros(fs)
sinal_5[5000] = 1.0

# Teste 6: Constante
sinal_6 = np.ones(fs)

def testar_sinal(nome, sinal_teste):
    print(f"--- {nome} ---")
    wp_test = pywt.WaveletPacket(sinal_teste, 'db4', 'symmetric',
        maxlevel=4)
    rec = wp_test.reconstruct(update=False)
    validar_reconstrucao(sinal_teste, rec)

testar_sinal("Duas Senoides", sinal_2)
testar_sinal("Ruido Gaussiano", sinal_3)
testar_sinal("Senoide + Ruido", sinal_4)
testar_sinal("Impulso", sinal_5)
testar_sinal("Constante", sinal_6)
```

Capítulo 7

Experiência Caixa Preta

Abaixo temos um script completo que engloba tudo, servindo como uma "caixa de areia"(sandbox) para você testar mudanças.

Listing 7.1: Caixa Preta Completa

```
import numpy as np
import pywt
import time

# PARAMETROS CONFIGURAVEIS
FREQUENCIA = 440
SAMPLE_RATE = 16000
DURACAO = 1.0
WAVELET = 'db4'
NIVEL = 4

# GERA SINAL
t = np.arange(0, DURACAO, 1/SAMPLE_RATE)
sinal_original = np.sin(2 * np.pi * FREQUENCIA * t)

# DECOMPOSICAO
inicio_dec = time.time()
wp = pywt.WaveletPacket(sinal_original, wavelet=WAVELET, maxlevel=
    NIVEL)
fim_dec = time.time()

nos = wp.get_level(NIVEL, order='freq')

print(f"=== SINTESE ===")
print(f"Tempo Decomposicao: {(fim_dec-inicio_dec)*1000:.2f} ms")
print(f"Total de nos: {len(nos)}")

energias = []
for i, no in enumerate(nos):
    energia = np.sum(no.data**2)
    energias.append(energia)
    print(f"Sub-banda {i}: tam={len(no.data)}, energia={energia:.2f
        }")

```

```
# RECONSTRUCAO GERAL
inicio_rec = time.time()
sinal_rec = wp.reconstruct(update=False)
fim_rec = time.time()

print(f"Tempo Reconstrucao: {(fim_rec-inicio_rec)*1000:.2f} ms")
print(f"Erro max reconstrucao: {np.max(np.abs(sinal_original -
    sinal_rec)):.2e}")
```

Capítulo 8

Funções Reutilizáveis

Neste ponto, organizaremos códigos limpos com *docstrings* para compor o nosso arquivo `utils_wpt.py`.

Listing 8.1: Módulo utils_wpt.py

```
import numpy as np
import pywt

def decompor_wpt(sinal, wavelet='db4', nivel=4, modo='symmetric'):
    """Decompoe um sinal via WPT e retorna a arvore."""
    return pywt.WaveletPacket(data=sinal, wavelet=wavelet, mode=
        modo, maxlevel=nivel)

def reconstruir_wpt(wp):
    """Reconstroi sinal original a partir da arvore."""
    return wp.reconstruct(update=False)

def reconstruir_subbanda(wp_original, caminho_no):
    """Cria uma WP vazia, preenche um unico caminho de no e
        reconstroi para tempo."""
    wp_vazia = pywt.WaveletPacket(data=None, wavelet=wp_original.
        wavelet, mode=wp_original.mode)
    wp_vazia[caminho_no] = wp_original[caminho_no].data
    return wp_vazia.reconstruct(update=False)

def calcular_energia(dados):
    """Calcula energia simples de um array."""
    return np.sum(np.power(dados, 2))
```

Capítulo 9

15 Desafios Práticos

1. Mude o nível de decomposição (`maxlevel`) de 4 para 5 e observe o número de sub-bandas.
2. Troque a wavelet para `db8` e verifique se o tamanho do array resultante aumenta.
3. Troque a wavelet para `haar` e compare o tempo de execução.
4. Calcule a FFT da "Banda 0"reconstruída no tempo para comprovar que contém a banda correta de frequência.
5. Gere um sinal de áudio `chirp` (varredura de frequência de 0 a 8000 Hz) e plote as energias de todas as 16 bandas no tempo.
6. Reconstrua apenas as bandas ímpares e ouça/verifique o sinal (se for áudio).
7. Tente forçar um `maxlevel=15` no sinal de 16000 amostras. O que o PyWavelets diz?
8. Mude o sinal para ter 16001 amostras. Verifique o impacto no tamanho da árvore WPT e reconstrução.
9. Escreva uma função que encontre em qual banda está a frequência de 6500 Hz.
10. Inverta (multiplique por -1) os coeficientes da banda 0 e reconstrua. Observe o sinal no tempo.
11. Zere todas as bandas acima da banda 7 e reconstrua. Você acabou de fazer um filtro passa-baixas!
12. Faça o gráfico do Erro de Reconstrução Variando o modo de extensão.
13. Plote a sub-banda extraída contra a senoide original filtrada via SciPy.
14. Compare o custo computacional entre $L = 3, 4, 5, 6$. Use `time.time()`.
15. (Desafio Difícil) Encontre a taxa real de amostras *redundantes* adicionadas por *padding* por banda.

Capítulo 10

Referencial Teórico — Template

Este é o esqueleto para seu **Referencial Teórico** focado nas técnicas utilizadas no PIBITI:

- **1. Introdução a Deepfakes de Áudio e o Dataset ASVspoof**
  - O que são ataques lógicos (TTS, Voice Conversion).
  - Estrutura básica do ASVspoof (Bonafide vs Spoof).
- **2. Transformada de Wavelet e WPT**
  - Diferença entre Fourier e Wavelet (resolução tempo-frequência).
  - Por que Wavelet Packet (WPT) e não apenas a árvore DWT tradicional? (necessidade de resolução nas altas frequências).
  - Famílias de wavelets adequadas para áudio.
- **3. Descritores de Energia e Contorno de Onda**
  - Teorema de Parseval para energias.
  - Extração de envelope/contorno e energia por sub-banda.
- **4. Classificadores Clássicos vs Deep Learning**
  - (Breve nota sobre SVM, Random Forest que utilizaremos adiante versus modelos ponta-a-ponta).
- **5. Métricas de Avaliação**
  - EER (Equal Error Rate).
  - Min t-DCF.

Capítulo 11

Checklist do Mês 3

Rascunho 1 do Referencial Teórico entregue.

`pywt` e dependências instalados no ambiente virtual.

Sinal de 440 Hz e Ruído gerados para testes.

Decomposição `db4` em 16 nós (L=4) operacional e compreendida.

Reconstrução do sinal inteiro validada com erro na ordem de 10^{-10} ou menor.

Reconstrução de sub-bandas de frequência individuais funcionando.

Funções modulares criadas e testadas num script de validação ("Caixa Preta").

Capítulo 12

Desafio Final do Mês 3

Desafio

Crie um sinal compostos de TRÊS senoides (ex: 200 Hz, 1500 Hz e 6000 Hz). Decomponha com WPT no nível adequado. Isole a banda que deve conter APENAS os 1500 Hz. Reconstrua para o domínio do tempo e plote ao lado do sinal original de 1500 Hz. O erro entre ambos é aceitável? Se as bordas apresentarem distorção, **parabéns**, você acabou de ver os efeitos de vazamento de banda e resposta impulsiva do filtro na prática! Documente o achado.